Vulnerability Assessment Report

Matthew Jordan Jeftha **CIN: FIT/MAR26/CS7242**

Date: 26 March 2026

Target Website: https://demo.testfire.net

I performed a vulnerability assessment on a web application using OWASP ZAP, Nmap, and browser tools. Several security issues were identified, including missing security headers and cookie vulnerabilities. These issues could expose the system to attacks and should be fixed.

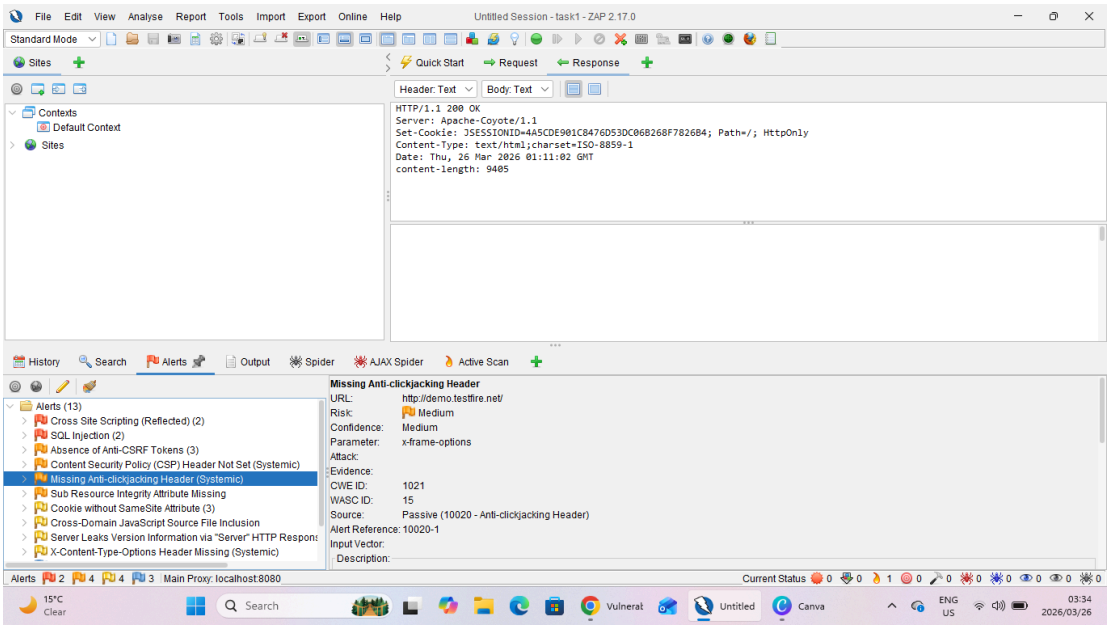
Tools Used

- OWASP ZAP
- Nmap
- Chrome DevTools

Findings:

Vulnerability 1: Missing Anti-clickjacking Header

- **Description:** The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-frame-Options.
- **Risk:** Medium
- **Impact:** An attacker can embed the site in an iframe, potentially tricking uers into performing unintented actions (clickjacking)
- **Evidence:**

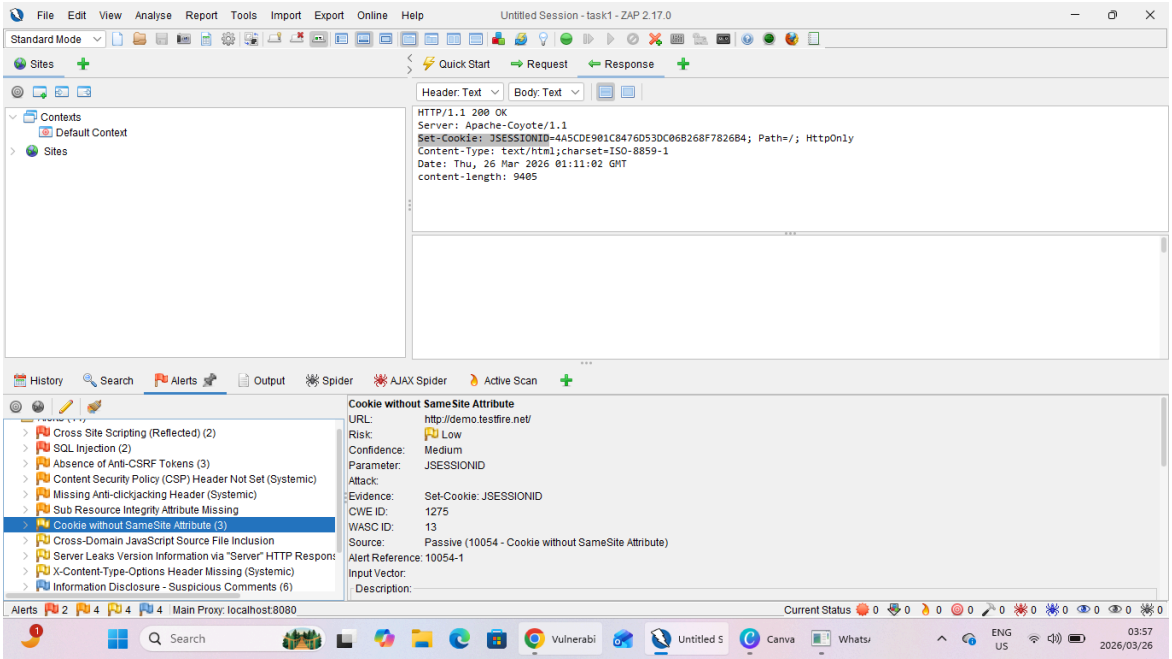


- **Fix:** Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page

to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

Vulnerability 2: Cookie without SameSite Attribute

- **Description:** A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a ‘cross-site’ request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.
- **Risk:** Low
- **Impact:** Cookies without SameSite are vulnerable to CSRF attacks. Set SameSite to lax or strict to mitigate risks.
- **Evidence:**

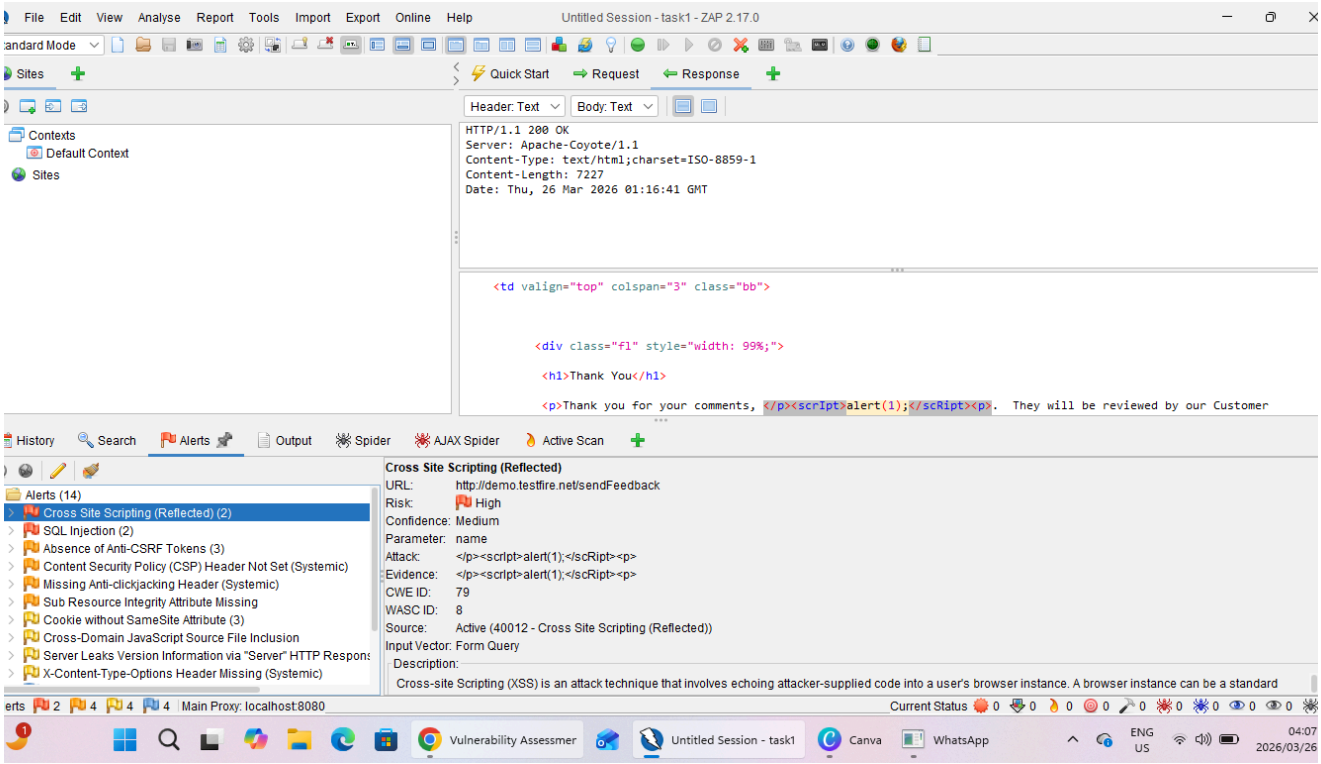


- **Fix:** Ensure that the SameSite attribute is set to either lax or ideally strict for all cookies.

Vulnerability 3: Cross Site Scripting (Reflection)

- **Description:** When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks are typically exploited to steal session cookies, perform unauthorized actions on behalf of the user, or deface web pages.
- **Risk:** High
- **Impact:** failing to use proper output encoding is that user-supplied data can be interpreted as executable code, leading to vulnerabilities like cross-site scripting (XSS) or injection attacks. This can allow attackers to steal sessions, deface web pages, or execute malicious scripts in the victim’s browser, compromising both data integrity and user trust.

- **Evidence:**



- **Fix:**

Phases:Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module and Apache Wicket

Phases: implementation and Architecture and Design

Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies.

.